

FoodBridge — React Frontend Mentorship

Day 5 Notes — Role-Aware Navbar, Logout, Redirects, Dashboards

0. Revision Recap (Warm-up)

- localStorage persists the JWT across page refreshes — solves the 'refresh logs you out' problem.
- useEffect in AuthProvider is still required because localStorage alone doesn't update React state automatically — the app must explicitly read localStorage and call setUser() to load it into live state.
- We decode the JWT (not encode) because the raw token is encoded and unreadable directly — decoding extracts usable fields like role and email (sub).

1. Public vs Protected Pages (Design Decision)

Confirmed that Home/About/Contact should remain visible to everyone, even logged-out users — same pattern as real platforms (Swiggy, LinkedIn, Airbnb). These are informational/marketing pages meant to build trust before signup. Only functional/dashboard pages (Create Donation, My Donations, Admin panels, etc.) should be protected — this is the job of the upcoming ProtectedRoute component.

2. Role-Aware Navbar

Read user from context inside Navbar using the same useContext pattern as Login:

```
const { user, setUser } = useContext(AuthContext)
```

Conditional rendering based on user:

```
{!user && (  
  <>  
    <Link to="/login">Login</Link>  
    <Link to="/register">Register</Link>  
  </>  
)}  
  
{user && (  
  <>  
    <Link to={`/${user.role.toLowerCase()}/dashboard`} >Dashboard</Link>  
    <span>Welcome, {user.email}</span>  
    <button onClick={handleLogout}>Logout</button>  
  </>  
)}
```

New concept: Fragments (<>...</>)

A component/expression can only return ONE parent element in JSX. Fragments let multiple elements (e.g. Login + Register links) be grouped together without adding an extra, unwanted wrapper element like a <div> to the actual HTML output.

3. Logout Logic

```
const handleLogout = () => {  
  localStorage.removeItem("token")  
  setUser(null)  
}
```

```
}
```

Used `removeItem('token')` instead of `localStorage.clear()` — `clear()` wipes everything in `localStorage`, which could delete unrelated data stored later. `removeItem` targets only what this app owns.

Mistake caught: initially only cleared `localStorage` without calling `setUser(null)`. Since `localStorage` and React state are two separate storage locations, clearing one does NOT automatically update the other — the Navbar kept showing 'logged in' UI until a manual refresh. Both must be updated: `removeItem` for future persistence, `setUser(null)` for the current live UI.

4. Redirect After Login (useNavigate)

Until now, navigation only happened via the `Link` component, which requires a user click. `useNavigate` lets us change the route programmatically from inside JS code (e.g. right after a successful login).

```
import { useNavigate } from "react-router-dom"

const navigate = useNavigate()
...
navigate(`/${decoded.role.toLowerCase()}/dashboard`)
```

5. Recurring Bug: Reading State Right After Setting It

This exact bug appeared twice today. Both times, code tried to read 'user' (context state) immediately after calling `setUser(...)`, inside the same function — but state updates are not instant, so 'user' still held its OLD value (null) at that point.

```
// WRONG – user is still stale/null right after setUser()
setUser({ token, role: decoded.role, email: decoded.sub })
navigate(`/${user.role}/dashboard`) // fails or uses old value

// CORRECT – use the original source variable instead of state
setUser({ token, role: decoded.role, email: decoded.sub })
navigate(`/${decoded.role.toLowerCase()}/dashboard`)
```

Rule of thumb established: whenever data is needed IMMEDIATELY after calling a `set...` function, use the original source variable (here, `decoded`) instead of the state variable (`user`) — state is always one render behind at that point in the code.

Symptom observed from this bug: login required two clicks to redirect — first click updated context but crashed/failed navigation using stale null user; second click then navigated using the (now one-step-behind) previous state. Fixed by using `decoded.role` directly both times.

6. Placeholder Dashboard Pages

Created one placeholder page per role, using a 'folder per role' structure rather than 'folder per page' (different from Login/Register/Home pattern) — reasonable since each role will have multiple pages later (Dashboard, CreateDonation, MyDonations, etc. all living under one role folder).

```
src/pages/Donor/Dashboard.jsx
src/pages/Ngo/Dashboard.jsx
src/pages/Volunteer/Dashboard.jsx
src/pages/Admin/Dashboard.jsx
const DonorDashboard = () => {
  return (
    <div className="p-8">
      <h1 className="text-2xl font-bold">Donor Dashboard</h1>
    </div>
```

```
)  
}
```

```
export default DonorDashboard
```

Routes registered in App.jsx:

```
<Route path='/donor/dashboard' element={<DonorDashboard/>}/>  
<Route path='/ngo/dashboard' element={<NgoDashboard/>}/>  
<Route path='/volunteer/dashboard' element={<VolunteerDashboard/>}/>  
<Route path='/admin/dashboard' element={<AdminDashboard/>}/>
```

7. Styling Strategy Decision

Decided to keep building plain/unstyled UI until ALL core functionality is complete (Login, Register, all dashboards, donation lifecycle, protected routes, role-based access). A dedicated UI polish pass will happen afterward — better spacing/colors, icons (lucide-react), reusable Button/Input/Card components, loading states, and toast notifications. Reasoning: logic bugs are easier to spot in plain markup, and this mirrors real-world practice of building a functional MVP before investing in design.

8. Milestone Reached

Full authentication flow now works end-to-end: Register -> Login -> JWT decoded -> stored in context and localStorage -> Navbar updates instantly -> auto-redirect to correct role-based dashboard on first click -> logout instantly resets UI without refresh. Estimated roughly 30-35% through the full feature list.

9. Git Commit (End of Day 5)

```
git add .  
git commit -m "feat: role-aware Navbar, logout, redirect-after-login, and placeholder dashboards"
```

- Updated Navbar to conditionally render Login/Register vs Dashboard/Welcome/Logout based on user role
- Used Fragment (<>...</>) to group multiple conditional links without extra DOM wrapper
- Implemented handleLogout: removeItem('token') + setUser(null) for instant UI sync (no refresh needed)
- Added useNavigate for redirect-after-login to role-based dashboard route
- Fixed repeated state-timing bug: used decoded.role (fresh) instead of user.role (stale) for immediate redirect
- Created placeholder Dashboard.jsx for Donor, NGO, Volunteer, and Admin roles
- Registered all four dashboard routes in App.jsx
- Verified full flow: login redirects correctly on first click, logout works instantly, dashboard renders

10. Next Steps (Day 6)

- Build ProtectedRoute component for role-based route access (block wrong-role/unauthenticated access)
- Connect Register form to POST /users endpoint (same axios pattern as Login)
- Attach JWT automatically to protected requests via Axios interceptor
- Add auto-logout when token expires (using the exp field from decoded JWT)
- Add toast notifications (react-toastify) for login/register success and error feedback
- Start building real Donor Dashboard features (Create Donation form, My Donations list)